

Product Requirements Document

Industrial AI Twin

Edge AI Predictive Maintenance System for Machine Fault Detection

Module	COM683 Edge and Embedded Intelligence
Prepared for	Professor / Supervisor Review
Prepared by	Abdulrehman Mohamedrafiq Vohra
Version	v1.0
Date	15 June 2026
Document Type	Product Requirements Document (PRD)
Executive Positioning This project is defined as a lightweight industrial AI twin demonstrator: an embedded sensing node streams live machine condition data to an edge AI processor, where fault indicators, predictive trends, health scoring, and maintenance recommendations are shown through an interactive dashboard. The document deliberately separates confirmed prototype capabilities from future industrial expansion to keep the project technically credible.	

Document Control

Version	Date	Owner	Change Summary
1.0	15 June 2026	Abdulrehman Mohamedrafiq Vohra	Initial PRD for professor review and project scoping.

Static Contents

- 1. Product Vision and Context
- 2. Problem Statement
- 3. Goals, Non-Goals and Success Criteria
- 4. Stakeholders and User Groups
- 5. Product Scope
- 6. System Architecture
- 7. Hardware and Sensor Requirements
- 8. Data Pipeline and Processing Requirements
- 9. Machine Learning and Analytics Requirements
- 10. Fault Detection Requirements
- 11. Dashboard and User Experience Requirements
- 12. Functional Requirements
- 13. Non-Functional Requirements
- 14. Ethical, Social and Professional Considerations
- 15. Risks, Assumptions and Constraints
- 16. Validation and Testing Strategy
- 17. Roadmap and Future Expansion
- 18. Coursework Alignment
- 19. Open Questions for Professor Review

1. Product Vision and Context

The Industrial AI Twin is a real-time edge intelligence prototype for predictive maintenance. It creates a virtual representation of an industrial machine by combining live vibration and pressure telemetry, machine health analytics, fault state detection, future scenario forecasting, and maintenance recommendations. The system is intended for demonstration within COM683 Edge and Embedded Intelligence, with enough engineering depth to also support portfolio development.

Human Engineering View

The project should not be presented as a certified industrial diagnostic system. It should be presented as a technically credible prototype that demonstrates how embedded sensing and edge AI can support predictive maintenance decisions.

1.1 Product Summary

Area	Definition
Product Name	Industrial AI Twin
Domain	Industrial predictive maintenance / edge AI / embedded intelligence
Primary Objective	Detect and display early indicators of bearing wear, pressure anomaly, and impact events using real-time sensor data.
Core Hardware	Arduino Nano 33 BLE Sense, onboard accelerometer, pressure sensor, Raspberry Pi edge AI processor.
Core Software	BLE data ingestion, edge analytics, ML inference, dashboard, historical logging, rule-based fault engine.
Primary Output	Interactive dashboard showing current condition, fault state, predicted vibration trend, health score, and maintenance recommendation.
Target Users	Professor/supervisor, student developer, maintenance engineer persona, technical assessor.

2. Problem Statement

Industrial machines often show early signs of failure through subtle changes in vibration, pressure, and operating trends before a complete breakdown occurs. Traditional maintenance can be reactive or schedule-based, leading to unplanned downtime, unnecessary part replacement, safety risks, and increased operational cost. This project addresses the problem by demonstrating an edge-based system that continuously monitors machine condition and translates raw sensor signals into understandable fault states and maintenance actions.

2.1 Problem Context

- Bearing degradation, impact events, pressure leaks, and blockages can create measurable changes in machine behaviour.
- Industrial operators need early warnings that are understandable, not only raw charts.
- Cloud-only monitoring may introduce latency, dependency on connectivity, and privacy/security concerns.
- Edge AI allows local processing near the machine, enabling fast alerts and local decision support.
- For coursework, the challenge is to demonstrate the complete data-to-decision pipeline using constrained embedded/edge hardware.

3. Goals, Non-Goals and Success Criteria

3.1 Product Goals

- Build a working embedded-to-edge pipeline from Arduino sensor node to Raspberry Pi processing unit.
- Continuously capture vibration and pressure data and stream it over BLE.
- Extract meaningful features from raw sensor readings, including RMS, peak, moving average, kurtosis, trend slope, and pressure deviation.
- Detect three direct fault indicators: bearing wear risk, pressure anomaly, and impact event.
- Provide a dashboard that clearly communicates live condition, fault state, prediction, and recommended action.
- Demonstrate at least two machine learning components: linear regression for vibration forecasting and logistic regression for failure probability.
- Show responsible engineering judgement by documenting limitations, assumptions, ethical risks, and future work.

3.2 Non-Goals

Non-Goal	Reason
Certified industrial safety system	The project is a student prototype and will not be validated to industrial safety standards.
Full bearing defect frequency diagnosis	Requires shaft speed, bearing geometry, high-frequency sampling, and controlled test rigs.
Cloud-scale predictive maintenance platform	The coursework focus is edge and embedded intelligence, not enterprise cloud monitoring.
Guaranteed 24-hour Remaining Useful Life prediction	The project can forecast trend scenarios, but not guarantee real RUL without long-term labelled industrial data.
All possible machine faults	Rotor imbalance, misalignment, looseness, cavitation, and motor electrical faults are future expansion items.

3.3 Success Criteria

ID	Success Criterion	Measurement
SC-01	Sensor node streams live data to edge device	Stable BLE data feed for continuous demo period.
SC-02	Dashboard updates in near real time	Visible chart/gauge updates within acceptable latency.
SC-03	Bearing wear indicator is triggered correctly in controlled test	Simulated/injected rising vibration and kurtosis threshold produces critical status.
SC-04	Pressure anomaly is shown distinctly	Deviation from calibrated baseline creates yellow warning state and recommendation.
SC-05	Impact event self-clears	Short vibration spike is logged and visualised without permanent false failure state.
SC-06	Prediction module produces 24-hour scenarios	Best, likely, and worst-case vibration forecast is displayed.
SC-07	Professor can understand system value quickly	PRD, architecture, demo flow, and dashboard are clear and defensible.

4. Stakeholders and User Groups

Stakeholder/User	Needs	Project Relevance
Professor / Marker	Clear problem, justified hardware, ML method, challenges, working demo.	Main assessment audience.
Maintenance Engineer Persona	Fast understanding of machine status and recommended actions.	Guides dashboard and alert design.
Student Developer	Feasible implementation within time and available equipment.	Controls scope and technical decisions.
Industrial Operator Persona	Simple active fault messages without needing to inspect raw data.	Justifies UX design.
Future Employer / Portfolio Viewer	Evidence of edge AI, embedded systems, ML, data pipeline, and professional documentation.	Supports CV/project showcase value.

5. Product Scope

5.1 In Scope

- Arduino Nano 33 BLE Sense sensor firmware for accelerometer and pressure readings.
- BLE communication from sensor node to Raspberry Pi.
- Raspberry Pi edge application for data ingestion, feature extraction, ML inference, and event detection.
- Interactive dashboard accessible locally over network or hosted locally on Raspberry Pi.
- Fault states for bearing wear risk, pressure anomaly, and impact event.
- Historical data storage for sensor readings and fault events.
- Calibration procedure for baseline vibration and pressure.
- Final demo script, technical documentation, and PRD.

5.2 Out of Scope for v1

- Certified integration with real industrial machinery.
- High-voltage motor control or actuator feedback.
- Cloud deployment unless time permits.
- Enterprise authentication/multi-user permissions.
- Advanced bearing frequency-domain diagnosis unless sampling and hardware support it.
- Full mobile application.

6. System Architecture

Recommended Architecture

Keep the architecture simple and defensible: embedded sensor node captures data, Raspberry Pi performs edge intelligence, dashboard displays the machine twin state. Do not overcomplicate the first version with cloud, Kubernetes, or enterprise services.

Layer	Component	Responsibility
Sensing Layer	Arduino Nano 33 BLE Sense	Collect accelerometer and pressure data.

Communication Layer	Bluetooth Low Energy	Transmit sensor packets to Raspberry Pi.
Edge Processing Layer	Raspberry Pi	Receive data, clean readings, extract features, run analytics/ML, classify events.
Storage Layer	SQLite / lightweight local database	Store sensor readings, features, predictions, and fault events.
Application Layer	Backend API / local service	Expose live state and history to dashboard.
Presentation Layer	Web dashboard	Display gauges, trends, fault cards, predictions, and recommendations.

6.1 Logical Data Flow

- Step 1: Arduino reads triaxial acceleration and pressure at defined sampling interval.
- Step 2: Arduino packages timestamp, acceleration axes, vibration magnitude, and pressure reading.
- Step 3: Raspberry Pi receives BLE packets and validates missing/corrupt samples.
- Step 4: Edge processor computes sliding-window features.
- Step 5: Fault rule engine evaluates thresholds, hysteresis, and event persistence.
- Step 6: ML modules estimate failure probability and forecast future vibration trend.
- Step 7: Dashboard receives live state and updates gauges, trend chart, status cards, and recommendations.
- Step 8: Data and events are stored for history, replay, and evaluation.

7. Hardware and Sensor Requirements

Hardware	Purpose	Notes / Challenge
Arduino Nano 33 BLE Sense	Embedded sensing node.	Board revision must be checked because IMU hardware differs across versions.
Triaxial accelerometer	Vibration sensing.	Good for prototype vibration magnitude and impact detection; frequency diagnosis may be limited by sampling rate.
Pressure sensor	Pressure anomaly indication.	Weakest part unless physical pressure setup is clearly defined; may need controlled airflow or external pressure transducer.
Raspberry Pi	Edge AI processor.	Runs data ingestion, feature extraction, ML inference, storage, and dashboard backend.
Test machine / rig	Fault demonstration source.	Could be small motor/fan rig with controlled vibration and induced disturbance.
Power supply and mounting	Stable demo setup.	Sensor mounting affects vibration quality; loose mounting may create false readings.

7.1 Sensor Sampling Requirements

Requirement	Target	Reason
Accelerometer sampling	Minimum 10-50 Hz for dashboard; ideally 100 Hz for better transient	Enough for trend/impact prototype, but not full industrial vibration

	detection.	spectrum analysis.
Pressure sampling	1-5 Hz or event-based average.	Pressure changes are slower than vibration.
Timestamping	Edge timestamp plus sensor packet counter.	Supports missing sample detection and historical analysis.
Calibration period	2-5 minutes baseline collection.	Required for pressure baseline and normal vibration level.

8. Data Pipeline and Processing Requirements

8.1 Raw Data Fields

Field	Type	Description
timestamp	datetime	Time reading is received or generated.
ax, ay, az	float	Raw acceleration values from three axes.
vibration_magnitude	float	Computed magnitude from triaxial acceleration.
pressure_hpa	float	Pressure reading in hPa.
temperature_optional	float	Optional temperature if available/used for context.
device_id	string	Sensor node identifier.
packet_seq	integer	Packet sequence for missing sample detection.

8.2 Feature Engineering

Feature	Method	Used For
Vibration RMS	Root mean square over sliding window.	General vibration severity and health score.
Peak vibration	Maximum value over window.	Impact/spike detection.
Kurtosis	Statistical measure of impulsiveness.	Bearing wear indicator and impact event detection.
Moving average	Rolling average over recent readings.	Trend smoothing and dashboard chart.
Trend slope	Regression/slope over recent window.	Bearing wear trend and forecast input.
Pressure deviation	Current pressure minus calibrated baseline.	Leak/blockage anomaly indicator.
Fault persistence timer	Tracks how long threshold is exceeded.	Avoids false alarms from single noisy readings.

9. Machine Learning and Analytics Requirements

Important Wording Correction

Do not call health scoring and trend analysis machine learning models unless they are implemented as trained models. In v1, present them as analytics modules. This keeps the technical explanation honest and stronger.

Module	Type	Input	Output	Purpose
Linear Regression Forecast	ML/statistical model	Historical vibration features and time index.	Best, likely, worst-case 24-hour vibration forecast.	Predict trend direction for digital twin future scenario.
Logistic Regression Failure Probability	ML classifier	RMS, kurtosis, trend slope, pressure deviation, spike count.	Failure probability percentage.	Estimate machine risk level.
Health Score Engine	Rule/score algorithm	Fault states, vibration severity, pressure deviation, trend.	0-100 health score.	Explainable machine condition score.
Trend Analysis Engine	Analytics module	Moving window sensor history.	Slope, direction, degradation rate.	Support bearing wear and forecast logic.
Fault Rule Engine	Deterministic logic	Thresholds, persistence, model outputs.	Active fault status.	Convert features into dashboard messages.

9.1 Model Training Strategy

- Use a small labelled or semi-labelled dataset generated from controlled operating conditions: normal, increased vibration, pressure deviation, and impact spikes.
- If real labelled data is limited, clearly state that controlled fault simulation is used for prototype validation.
- Split data into training and test segments where possible, even if the dataset is small.
- Evaluate logistic regression using accuracy, confusion matrix, precision/recall, and false alarm discussion.
- Evaluate forecasting using visual trend quality and simple error metric such as MAE where appropriate.
- Keep models lightweight enough to run on Raspberry Pi with low latency.

10. Fault Detection Requirements

10.1 Fault Definitions

Fault	Detection Logic	Dashboard Behaviour	Engineering Challenge
Bearing Wear Risk	Increasing vibration trend plus elevated kurtosis above 3.5 and persistence over a time window.	Red status: CRITICAL - Bearing Wear Risk Detected; vibration gauge turns red.	Avoid overclaiming actual bearing damage without controlled bearing data.
Pressure Anomaly	Pressure deviation from calibrated baseline greater than 10 hPa.	Yellow pressure gauge; warning: Pressure anomaly - check for leaks or blockages.	Must justify what pressure source is being measured.
Impact Event	Sudden vibration spike with kurtosis above 4.0 and short duration.	Temporary red spike and event log entry; auto-clears if signal returns to normal.	Needs self-clearing logic to avoid false permanent fault.

10.2 Alarm State Rules

State	Meaning	Trigger	Clear Condition
NORMAL	Machine appears stable.	Features within calibrated baseline range.	N/A
WARNING	Early anomaly or pressure deviation.	Pressure deviation or mild vibration trend.	Return to normal for defined period.

CRITICAL	High-risk fault indication.	Bearing wear risk or severe vibration condition persists.	Manual acknowledgement or sustained return to safe range.
TRANSIENT	Temporary impact/spike event.	Short high-kurtosis vibration spike.	Auto-clear after signal normalises; event remains logged.

11. Dashboard and User Experience Requirements

11.1 Dashboard Components

Component	Purpose	Required Behaviour
Machine Status Header	Immediate high-level state.	Shows Normal, Warning, Critical, or Transient.
Vibration Gauge	Shows current vibration severity.	Green/yellow/red colour mapping with threshold legend.
Pressure Gauge	Shows pressure value and anomaly state.	Yellow warning on deviation greater than threshold.
Real-Time Trend Chart	Displays recent vibration over time.	Updates continuously and highlights spikes.
Prediction Chart	Shows future vibration scenarios.	Best, likely, worst-case over 24 hours.
Health Score Card	Summarises machine condition.	0-100 score with explanation.
Fault Cards	Displays direct fault messages.	Bearing wear, pressure anomaly, impact event.
Recommendation Panel	Translates fault into action.	AI/rule-generated maintenance recommendation.
Event Log	Records detected events.	Timestamp, fault type, severity, duration, status.

11.2 UI Severity Colours

Severity	Colour	Meaning
Normal	Green	Machine readings are within baseline range.
Warning	Yellow/Amber	Anomaly or early risk requires inspection.
Critical	Red	High-risk condition or detected serious fault indicator.
Transient	Red pulse / temporary marker	Short-lived impact event that may self-clear.

12. Functional Requirements

ID	Area	Requirement	Priority
FR-01	Sensor Reading	The Arduino shall collect triaxial acceleration data from the accelerometer.	Must
FR-02	Pressure Reading	The Arduino shall collect pressure readings from the	Must

		pressure sensor.	
FR-03	BLE Transmission	The Arduino shall transmit readings to Raspberry Pi using BLE.	Must
FR-04	Data Validation	The edge application shall detect missing, invalid, or delayed packets.	Should
FR-05	Feature Extraction	The edge application shall compute RMS, peak, kurtosis, moving average, trend slope, and pressure deviation.	Must
FR-06	Baseline Calibration	The system shall establish normal vibration and pressure baselines before fault detection.	Must
FR-07	Bearing Wear Risk	The system shall show bearing wear risk when rising vibration trend and high kurtosis persist.	Must
FR-08	Pressure Anomaly	The system shall show pressure anomaly when pressure deviates beyond threshold.	Must
FR-09	Impact Event	The system shall detect sudden spikes and self-clear transient alerts.	Must
FR-10	Forecasting	The system shall forecast best, likely, and worst-case vibration scenario over 24 hours.	Should
FR-11	Failure Probability	The system shall estimate failure probability using a lightweight model.	Should
FR-12	Health Score	The system shall show a machine health score from 0 to 100.	Should
FR-13	Recommendation	The dashboard shall show maintenance recommendations based on active fault state.	Must
FR-14	Historical Storage	The system shall store readings and events for review.	Should
FR-15	Demo Mode	The system should support simulated/injected fault scenarios for reliable demonstration.	Should

13. Non-Functional Requirements

Category	Requirement	Target
Latency	Dashboard should update quickly enough for real-time demonstration.	Prefer under 2 seconds end-to-end.

Reliability	System should survive short BLE dropouts.	Show connection status and resume cleanly.
Explainability	Fault decisions should be understandable.	Show which threshold/feature contributed to alert.
Maintainability	Code should be modular.	Separate sensor, BLE, feature extraction, ML, dashboard, and configuration.
Usability	Dashboard should be clear to non-experts.	Status first, details second.
Portability	System should run locally on Raspberry Pi.	Minimal external dependency.
Safety	System should not control machinery.	Monitoring and decision support only.
Data Integrity	Stored readings should include timestamps and device IDs.	Supports evidence and demo replay.

14. Ethical, Social and Professional Considerations

Consideration	Risk	Mitigation
False Positives	Unnecessary maintenance or reduced trust in system.	Use persistence windows, confidence scores, and clear event logs.
False Negatives	Missed fault may create safety or cost risk.	State limitations and keep system as decision support, not safety-critical control.
Human Trust	Users may overtrust AI recommendations.	Explain feature contribution and confidence level.
Workplace Impact	Automation can change maintenance roles.	Frame system as assisting engineers, not replacing them.
Data Privacy/Security	Industrial telemetry may reveal operational patterns.	Prefer local edge processing and document network security considerations.
Professional Responsibility	Prototype may be mistaken for certified diagnostic system.	Use careful wording and clear limitations in report/demo.
Sustainability	Predictive maintenance may reduce waste and downtime.	Connect to SDG 9 Industry, Innovation and Infrastructure and SDG 12 Responsible Consumption.

15. Risks, Assumptions and Constraints

15.1 Key Risks

Risk	Impact	Likelihood	Mitigation
Pressure sensor does not represent real industrial pressure.	High	Medium	Define controlled pressure demo or use external pressure transducer.
BLE instability causes missing data.	Medium	Medium	Add connection state, packet counter, retry/reconnect logic.
Vibration data too noisy due to poor mounting.	High	Medium	Use firm mounting, calibration, smoothing, and repeated tests.

ML dataset too small.	High	High	Use controlled scenario data and be honest about prototype validation.
Too much scope for available time.	High	Medium	Prioritise core demo: live sensor, three faults, dashboard, logging.
Overclaiming digital twin capability.	Medium	Medium	Define as lightweight AI twin demonstrator with live/current/predicted state.
Sampling rate insufficient for advanced vibration analysis.	Medium	Medium	Use time-domain features for v1; list FFT/frequency diagnosis as future work.

15.2 Assumptions

- Arduino Nano 33 BLE Sense hardware is available and compatible with required sensor libraries.
- Raspberry Pi can run Python services, BLE client, ML inference, storage, and dashboard stack.
- A small physical test rig or controlled demo method will be available to create normal, vibration, pressure, and impact conditions.
- The project will be assessed as a prototype demonstration, not as a certified industrial system.
- The professor accepts Raspberry Pi as the edge AI processor; TinyML on Arduino may be considered optional enhancement.

16. Validation and Testing Strategy

Test Area	Test Method	Pass Criteria
Sensor Readings	Print/log accelerometer and pressure values during normal operation.	Values are stable and physically plausible.
BLE Link	Run continuous transmission test.	No major dropouts during demo window.
Feature Extraction	Use known sample data to verify RMS, peak, kurtosis, and slope.	Computed values match expected calculations.
Bearing Wear Scenario	Gradually increase vibration or replay labelled data.	Critical bearing wear risk appears only after persistence condition.
Pressure Scenario	Introduce or simulate pressure deviation.	Yellow pressure anomaly appears with correct recommendation.
Impact Scenario	Tap/impact machine or inject spike.	Temporary spike appears and self-clears.
Forecasting	Feed increasing, stable, and decreasing trend data.	Prediction chart direction matches trend.
Dashboard UX	Show to professor/peer and ask them to identify current machine state.	User understands status within 10 seconds.

17. Roadmap and Future Expansion

Phase	Scope	Outcome
MVP Phase	Live sensor stream, BLE, dashboard, vibration/pressure gauges.	Basic working IoT monitoring system.
Fault Detection Phase	Feature extraction, three fault states, event log.	Clear predictive maintenance behaviour.

AI Twin Phase	Health score, failure probability, 24-hour forecast, recommendations.	Stronger edge AI demonstration.
Robustness Phase	Calibration, hysteresis, confidence, validation tests.	Reduced false alarms and stronger report evidence.
Future Work	Rotor imbalance, misalignment, looseness, FFT, external transducers, cloud sync, TinyML.	Portfolio-grade expansion path.

17.1 Future Fault Types

Future Fault	Likely Indicators	Reason Deferred
Rotor Imbalance	Dominant vibration at rotational frequency.	Needs speed/frequency analysis.
Misalignment	Harmonics and directional vibration patterns.	Needs more detailed vibration analysis and test rig.
Mechanical Looseness	Impulsive vibration and broad spectrum components.	May be confused with impact events without richer data.
Cavitation / Flow Issues	Pressure fluctuation and vibration noise.	Needs appropriate pump/fluid setup.
Motor Electrical Fault	Current signature and thermal behaviour.	Requires additional sensors.

18. Coursework Alignment

Assessment Fit

The assignment brief asks for a real-world embedded intelligence solution, including problem overview, existing research, sensors/hardware, ML approach, ethical/social/technical challenges, references, and presentation quality. This PRD has been structured to support those exact areas.

Coursework Area	How This Project Aligns	Strength
Problem Overview	Addresses industrial downtime, predictive maintenance, safety, and cost reduction.	Strong
Existing Research	Supports literature review around predictive maintenance, vibration analysis, edge AI, TinyML, and digital twins.	Strong if properly researched
Sensors/Hardware	Uses embedded sensing, accelerometer, pressure sensor, BLE, and Raspberry Pi edge processing.	Strong, but pressure setup must be justified
Data Collection, Processing and ML	Includes live data, features, regression, classification, health scoring, trend analysis.	Very strong
Ethical, Social and Technical Challenges	Covers false alarms, safety, trust, limitations, and local processing.	Strong if discussed critically
References	Can use academic papers and high-quality sources for vibration diagnosis, edge AI, digital twins, BLE, and predictive maintenance.	Needs work
Presentation Quality	Dashboard, architecture diagram,	Strong

scenario demo, and PRD support a clear pitch.

19. Open Questions for Professor Review

Question	Why It Matters
Should ML inference run only on Raspberry Pi, or should one TinyML model also run on Arduino?	This affects how strongly the project demonstrates embedded ML versus edge ML.
Is the pressure anomaly expected to be real physical pressure, simulated pressure, or environmental pressure?	This determines whether LPS22HB is acceptable or an external pressure sensor is needed.
What level of real machine rig is expected for assessment?	Controls project feasibility and hardware risk.
Are controlled/simulated fault scenarios acceptable for model training and demo?	Important because real labelled industrial faults are difficult to obtain.
Should the final dashboard be local-only or web-accessible over LAN?	Affects implementation and demo setup.
How much emphasis should be placed on literature review versus prototype build?	Helps balance documentation and coding effort.

Appendix A: Recommended Final Project Statement

Professor-Facing Statement

This project proposes a lightweight Industrial AI Twin for edge-based predictive maintenance. An Arduino Nano 33 BLE Sense collects vibration and pressure telemetry and streams it to a Raspberry Pi edge processor over BLE. The edge processor extracts statistical features, applies lightweight ML and rule-based logic, and presents machine health, fault indicators, future vibration scenarios, and maintenance recommendations through an interactive dashboard. The v1 prototype focuses on bearing wear risk, pressure anomaly, and transient impact events, while documenting limitations and future expansion into more advanced fault classes such as imbalance, misalignment, and looseness.

Appendix B: Recommended Reference Areas

- Edge AI and embedded machine learning for predictive maintenance.
- Vibration analysis and statistical features such as RMS, kurtosis, skewness, and peak factor.
- Digital twins for industrial condition monitoring.
- BLE communication in sensor networks.
- TinyML and embedded inference on constrained devices.
- Ethical and professional challenges in AI-based maintenance decision support.
- UN Sustainable Development Goals related to industry, innovation, infrastructure, and responsible production.